

Java/script: no alnum cheat sheets

Java/script: no alnum cheat sheets - Author not stated, sla.ckers.org.

- Published: date not stated
- Original: <<http://sla.ckers.org/forum/read.php?24,33349>>
- Preserved from: <http://sla.ckers.org/forum/read.php?24,33349> (stored) on 2026-08-09
- Capture timestamp: 20120211062554
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Java/script: no alnum cheat sheets

[[[image: Cenzic 232 Patent](#)]](<http://bit.ly/vEaqkw>) Paid Advertising

sla.ckers.org is [ha.ckers](#) sla.cking

[[[image: Sla.ckers.org](#)]](<http://sla.ckers.org/forum/index.php>)

Script obfuscation, filter evasion, IDS/IPS/WAF bypassing... this is where it should live. Because this topic is too big to live anywhere else. Phj33r!

Java/script: no alnum cheat sheets

Posted by: * [SW](#) *

Date: February 09, 2010 09:12PM

Cheat sheets of the shortest ways we can find to accomplish things with different no alnum charsets.

Feel free to fill in different charsets and of course if you find a shorter version of a letter post it.

Charset: + (seemingly impossible to execute with)

0: 3: +[]

1: 11: ++[] [+]

2: 20: ++[+[] [+]] [+]

3: 29: ++[+ [+[] [+]] [+]] [+]

10: 17: ++[] [+] [+]

undefined: 6: [] []

Infinity: 86: +(+[[] [+]] +([+[] []] +[] [])) ++[] [+] [+] ++[[] [+]] +[] +[] +[] +[]

NaN: 7: +[] []

a: 25: "NaN"[1]

(+[] [] +[]) ++[] [+]

b: :

c: :

d: 33: "undefined"[2]

([] [] +[]) ++[+[] [+]] [+]

e: 37: "NaNUndefined"[10]

([+[] []] +[] []) ++[] [+] [+]

f: 51: "undefined"[4]

([] [] +[]) ++[+ [+ [+[] [+]] [+]] [+]] [+]

g: :

h: :

i: 60: "undefined"[5]

([] [] +[]) ++[+ [+ [+ [+[] [+]] [+]] [+]] [+]] [+]

j: :

k: :

l: :

m: :

n: 24: "undefined"[1]

([] [] +[]) ++[] [+]

o: :

p: :

q: :

r: :

s: :

t: :

u: 16: "undefined"[0]

([] [] +[]) [+]

v: :

w: :

x: :

y: 108: "NaNInfinity"[10]

(+(![] +(+([[] [+]] +([+[] []] +[] [])) ++[] [+] [+] ++[[] [+]] +[] +[] +[] +[])) +!+[] +[]

Z: :

A: :

B: :

C: :

D: :

E: :

F: :

G: :

H: :

I: 94: "Infinity"[0]

(+(+[[]][+[]]+([+[][]]+[][]))++[[]][+[]]+[+[]]+[[+[]][+[]]+[+[]]+[+[]]+[+[]])+[])(+[[]]

J: :

K: :

L: :

M: :

N: 17: "NaN"[0]

(+[[][]]+[])(+[[]]

O: :

P: :

Q: :

R: :

S: :

T: :

U: :

V: :

W: :

X: :

Y: :

Z: :

Charset: +! (six with !)

0: 3: +[]

1: 5: +!+[]

2: 9: !+[]+!+[]

3: 14: !+[]+!+[]+!+[]

10: 11: +!+[]+[+[]]

```
undefined: 6: [[]]
```

Infinity: 60: $+(+!+[]+(!+[[]])![+[]+!+[]+!+[]]+[+!+[]]+[+[]]+[+[]]+[+[]])$

NaN: 6: +[![]]

```
true: 4: !![]
```

false: 3: ![]

```
a: 15: "false"[1]
```

$$(![]+[])[+!+[]]$$

```
b: 424: ([["sort"],["call"])()+[2]
```

[illegible]

```
c: 144: ([["filter"]]+[ ])[3]
```

$$(\square((\neg\square+\square)[+\square]+(\neg\square)+\square[\square])(+!+\square+[\square])+(\neg\square+\square)!+\square+!+\square)+(!+\square+\square)[+\square]+(!+\square+\square)[!+\square+!+\square+!+\square]+(!+\square+\square)[!+\square]+(\square)[!+\square+!+\square+!+\square]$$

```
d: 22: "undefined"[2]
```

$$(\square[\square]+\square)[!+\square+!+\square]$$

```
e: 27: "true"[3]
```

$$(!+[]+[])(!+[]+!+[]+!+[])$$

```
f: 13: "false"[0]
```

$$(![]+[])[+[]]$$

g. :

```
h: : [{"sort"}["call"]()["atob"]("aN")[0]
```

```
i: 27: "falseundefined"[10]
```

$$([![]]+[][]) [+!+[]+[+[]]]$$

```
j: 429: ([["sort"],["call"])()+[3]
```

[illegible] $k:$

```
l: 19: "false"[2]
```

$$(![]+[])(!+[]+!+[])$$

```
m: 730: (0["constructor"]+[])[11]
```

[illegible]

```
n: 18: "undefined"[1]
```

$$(\square[\square\square]+\square)[+!+\square]$$

```
o: 143: (true+[]["filter"])[10]
```

$$(!+[]+\square[(![]+\square)[+[]]+(![]+\square[\square])[+!+\square[+[]]]+(![]+\square)[!+\square+!+\square]+(!+\square+\square)[+[]]+(!+\square+\square)[!+\square+!+\square+!+\square]+(!+\square+\square)[+!+\square]))[+!+\square[+[]]]$$

p: :

 $q: \mathbb{R}^n \rightarrow \mathbb{R}^n$

```
r: 16: "true"[1]
```

$$(!![]+[])[+!+[]]$$

[illegible]

```
Y: : ["sort"]["call"]()["btoa"]("a")[0]
```

```
Z: : ["sort"]["call"]()["btoa"]("f")[0]
```

Charset: += (six with = bit longer but might be more useful if using 1 variable)

true: 9: ([])==+[])

false: 8: ([])==[])

a: :

b: :

c: :

d: :

e: :

f: 18: "false"[0]

(([])==[])+[])[+[]]

g: :

h: :

i: :

j: :

k: :

l: :

m: :

n: :

o: :

p: :

q: :

r: :

s: :

t: 19: "true"[0]

(([])==+[])+[])[+[]]

u: :

v: :

w: :

x: :

y: :

z: :

A: :

B: :

C: :

D: :

E: :

F: :

G: :

H: :

I: :

J: :

K: :

L: :
M: :
N: :
O: :
P: :
Q: :
R: :
S: :
T: :
U: :
V: :
W: :
X: :
Y: :
Z: :

Charset: **+!@/.,** (everything?)

a: :
b: :
c: :
d: :
e: :
f: :
g: :
h: :
i: :
j: :
k: :
l: :
m: :
n: :
o: :
p: :
q: :
r: :
s: :
t: :
u: :
v: :
w: :
x: :
y: :
z: :

A: :
B: :
C: :
D: :
E: :
F: :
G: :
H: :
I: :
J: :
K: :
L: :
M: :
N: :
O: :
P: :

Q: :
R: :
S: :
T: :
U: :
V: :
W: :
X: :
Y: :
Z: :

Edited 16 time(s). Last edit at 02/10/2010 08:23PM by SW.

Re: Java/script: no alnum cheat sheets

Posted by: * [LeverOne](#) *

Date: February 10, 2010 12:41AM

@SW

These ways are shortest:

1. Charset: +

"NaN": +[] []+[] "NaN"[1]: (+[] []+[])[++[] [+[]]

etc...

Infinity: +(++[] [+[]](+[] []+[] [])[++[] [+[]]+[] []]+++[] [+[]]+[] [+[]]+[] [+[]]) // 84

// [+[] []+[] [] - "NaNUndefined" // ([+[] []+[] [])[++[] [+[]]+[] []] // "e" or with only []+ Infinity: +
[++[] [+[]]+[] [+[] []+[] []][+[] ++[] [+[]]+[] []]+++[] [+[]]+[] [+[]]+[] [+[]] // 89

edit all! hahaha

Re: Java/script: no alnum cheat sheets

Posted by: * [SW](#) *

Date: February 10, 2010 01:24AM

Thanks very much Lever, updated them.

I think undefined[3] is shorter than NaNUndefined[10] if you have !.

Re: Java/script: no alnum cheat sheets

Posted by: * [LeverOne](#) *

Date: February 10, 2010 01:40AM

1. Charset: +!

Infinity: +(=!+[]+(!+[] [])[!+[]+!+[]+!+[]]+[!+[]]+[+[]]+[+[]]+[+[]]) // 60 NaN: +[![]] // 6

i: "falseundefined"[10] - ([![]]+[] [])[!+[]+[]] // 26

Thanks! :D

Edited 2 time(s). Last edit at 02/10/2010 08:46PM by LeverOne.

Re: Java/script: no alnum cheat sheets

Posted by: * [SW](#) *

Date: February 10, 2010 01:48AM

@ Lever After adding +[], NaN is 1 char shorter hmm! Infinity... well that's crazy! Nice finds.
updating again

OK, I see to get rid of the comma I forgot in there, using atob is now ~1048 characters, lol. Unless we have a shorter way to access window than `[["sort"]]["call"]`, without using comma. Seems quite infeasible without using variables.

Edited 1 time(s). Last edit at 02/10/2010 02:55AM by SW.

Re: Java/script: no alnum cheat sheets

Posted by: * *LeverOne* *

Date: February 10, 2010 05:57PM

Hi again!

If we need to get a String, we do not need intermediaries, because [NaN]+Infinity or NaN+[Infinity] is shorter than NaN+[]+Infinity

==> for example <==

 $// +$

```
y: 115: "NaNInfinity"[10] (+[][]) + [(++[])[+[]] + [(+[][]) + []][++[]][+[]] + [+[]] + [(++[])[+[]] + [+[]] + [+[]] + [+[]] + [+[]]) + [(++[])[+[]] + [+[]]]
```

// +!

```
y: 84: "NaNInfinity"[10] (+[![]]+(+(!+[]+(!+[]+[])[!+[]+!+[]+!+[]]+(!+[])+[+[]]+[+[]]+[+[]])))[!+[]+[]+[]]
```

```
// Maybe to recheck all letters? :D
```

Re: Java/script: no alnum cheat sheets

Posted by: * [SW](#) *

Date: February 10, 2010 08:25PM

You have a good point, I listed the values NaN, etc rather than strings "NaN" now.

I hope the other letters should be fine. Will work on more of the list tomorrow.

Re: Java/script: no alnum cheat sheets

Posted by: * [SW](#) *

Date: February 12, 2010 11:50AM

Strange.. [].constructor=[undefined], but [].constructor+[] = "function Array..."

Re: Java/script: no alnum cheat sheets

Posted by: * [SW](#) *

Date: February 12, 2010 08:15PM

I made a script to automatically expand something into the +! charset, no alnum.

It's simple, and probably buggy, and I didn't enter all the characters yet.

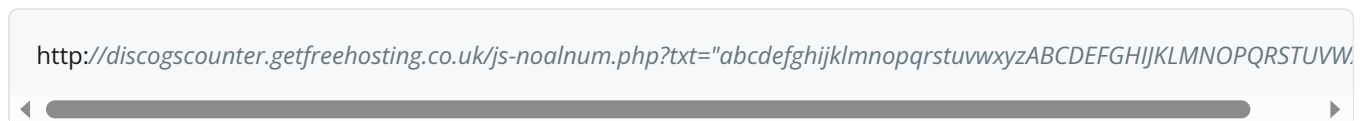
Check it, tell me errors or functions/objects missing (P.S. it only intends to convert normal, well-formed input).

<http://discogscounter.getfreehosting.co.uk/js-noalnum.php?txt=alert%28%22ecs%20ess%20ess%22%29>

Edit1: I see numbers are an issue, ie. `alert(100+4)` => 1004 because all these numbers are strings. Both `Number()` and `parseInt()` probably take about 1000 characters to generate.

Edit2: made php link

Edit3: Completed the alphabet. To print it (both cases) is over 35k long hmmm. :)



Edited 3 time(s). Last edit at 02/12/2010 10:39PM by SW.

Re: Java/script: no alnum cheat sheets

Posted by: * [LeverOne](#) *

Date: February 12, 2010 10:40PM

Good work, SW!

I notice a few imperfections:

- `[]['filter']['constructor']` is better than `[]['constructor']['constructor']`
- `[]['filter']constructor()` it is strategically more correct than `[]['sort']call"method"`, if we need to put all code into string.

exs.

`[]['sort']call"eval" // good`

`[]['filter']constructor") // is better`

Quote

I see numbers are an issue, ie. `alert(100+4)` => 1004 because all these numbers are strings

Maybe before to convert all "numbers" using "+" ? // `+( "100" )+4 --> +( +!+[]+[[]+[[]]] ) +!+[]+!+[]+!+[]+!+[] = 104`

upd:

Quote

Could you explain how it's strategically better to use filter.constructor

It is more comfortable for more difficult input-data. Yes, but size is a disadvantage.

Edited 1 time(s). Last edit at 02/13/2010 12:26AM by LeverOne.

Re: Java/script: no alnum cheat sheets

Posted by: * [SW](#) *

Date: February 12, 2010 11:27PM

Thanx for looking LeverOne.

Could you explain how it's strategically better to use filter.constructor than window.eval? I think they are around the same length, well, I don't mind which is used. :)

Good tip on the numbers, I will fix this and the nested quotes. One problem with using filter.constructor("eval(...)") is if you need quotes it will be quite long, I think it's like 500 characters to generate a ".

Re: Java/script: no alnum cheat sheets

Posted by: * [SW](#) *

Date: February 15, 2010 07:10AM

New version: http://discogscounter.getfreehosting.co.uk/js-noalnum_com.php?txt=alert%28%22XSS%22%29

Changes:

- no regex, recursive parsing, so no stepping
- added rest of letters and symbols
- allows single/double quotes, no nested
- deals with numbers vs. string numbers
- there are a few "optimizations" still to be worked out, both on the letters, and on the wrapping rules
- should be easily extensible to other charsets & to add variables

Edited 2 time(s). Last edit at 02/15/2010 07:25AM by SW.

Re: Java/script: no alnum cheat sheets

Posted by: * [.mario](#) *

Date: February 15, 2010 07:51AM

http://discogscounter.getfreehosting.co.uk/js-noalnum_com.php?txt=%3C/textarea%3E%3Cscript%3Ealert%28%22XSS%22%29%3C/script%3E

it's pointless - I know. Still ;)

g:0in~/for another/~alert(!1) (Å=",[Ç=!(μ=!Å+Å)+{}][Ç[ª=μ[++Å]+μ[Å-Å],È=Å-~Å]+Ç[È+È]+ª)]() [Ç[Å]+Ç[Å+Å]+μ[È]+ª](Å) [me](#) | | [PHPIDS](#) | | [Twitter](#) | | [SVGPurifer](#)

Sorry, only registered users may post in this forum.

[Click here to login](#)

Archived from <http://sla.ckers.org/forum/read.php?24,33349>. Rendered offline from the local Markdown copy.